

姓名: ***
学号: ***
班级: ***

《高级语言程序设计》大作业报告

*作品为个人编程和独立使用提示词操作 AI 原创，但本报告绝大部分内容由 AI 基于项目源代码生成

1. 项目介绍

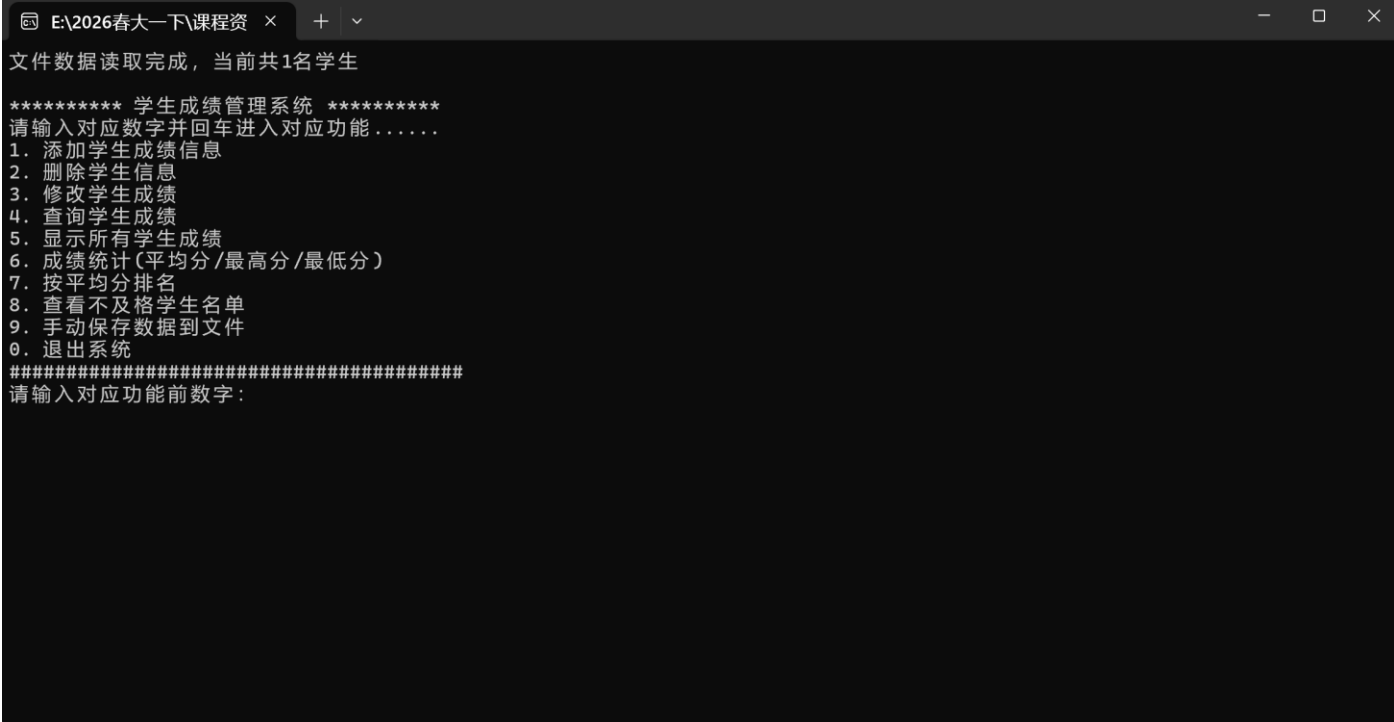
本程序基于 C++面向对象思想开发学生成绩管理系统，采用 vector 存储全体学生信息、map 存储单名学生多门课程成绩，搭配文件持久化存储，实现学生信息增删改查、成绩统计、排名、不及格筛选等完整业务功能。程序封装 Student 学生类、Manager 成绩管理类，代码模块化清晰，具备输入容错、文件读写、数据持久化等特性，可独立编译运行，满足课程设计作业要求。

2. 系统架构

本程序采用 vector 容器存储全体学生信息、map 容器存储单名学生多门课程成绩（<科目, 分数>映射），定义两个类——Student 类，存储学生信息成绩等；Manager 类，存储各种功能函数等，满足作业要求。

3. 部分功能展示

3.1 主程序界面



```
E:\2026春大一下\课程资 x + v
文件数据读取完成，当前共1名学生

***** 学生成绩管理系统 *****
请输入对应数字并回车进入对应功能.....
1. 添加学生成绩信息
2. 删除学生信息
3. 修改学生成绩
4. 查询学生成绩
5. 显示所有学生成绩
6. 成绩统计(平均分/最高分/最低分)
7. 按平均分排名
8. 查看不及格学生名单
9. 手动保存数据到文件
10. 退出系统
#####
请输入对应功能前数字:
```

按数字键后回车即可实现对应功能。

3.2 添加学生

```
E:\2026春大一下\课程资 × + ▾
文件数据读取完成，当前共1名学生

***** 学生成绩管理系统 *****
请输入对应数字并回车进入对应功能.....
1. 添加学生成绩信息
2. 删除学生信息
3. 修改学生成绩
4. 查询学生成绩
5. 显示所有学生成绩
6. 成绩统计(平均分/最高分/最低分)
7. 按平均分排名
8. 查看不及格学生名单
9. 手动保存数据到文件
0. 退出系统
#####
请输入对应功能前数字：1
===== 添加学生及成绩信息 =====
请输入学号：202500130130
请输入姓名：葛祎凡
请输入课程名称：高等数学
请输入成绩(成绩范围：0-100)：100
该学生存在，已对现有成绩更新完毕

按下回车以继续.....
```

3.3 查询学生个人成绩

```
E:\2026春大一下\课程资 × + ▾
请输入制定课程名称：1
暂无学生修读此课程

按下回车以继续.....4

***** 学生成绩管理系统 *****
请输入对应数字并回车进入对应功能.....
1. 添加学生成绩信息
2. 删除学生信息
3. 修改学生成绩
4. 查询学生成绩
5. 显示所有学生成绩
6. 成绩统计(平均分/最高分/最低分)
7. 按平均分排名
8. 查看不及格学生名单
9. 手动保存数据到文件
0. 退出系统
#####
请输入对应功能前数字：4
===== 查询学生成绩 =====
请输入学号：202500130130
---查询成功---
学号：202500130130姓名：葛祎凡
课程成绩：
程序设计基础：100；
高等数学：100；
高级语言程序设计：90；
总分：290平均分：96.6667

按下回车以继续.....
```

3.4 按平均分排名

```
E:\2026春大一下\课程资 × + v
请输入学号: 202500130001
请输入姓名: 苏厚诚
请输入课程名称: 高级语言程序设计
请输入成绩(成绩范围: 0-100): 50
新学生信息添加完毕

按下回车以继续.....

***** 学生成绩管理系统 *****
请输入对应数字并回车进入对应功能.....
1. 添加学生成绩信息
2. 删除学生信息
3. 修改学生成绩
4. 查询学生成绩
5. 显示所有学生成绩
6. 成绩统计(平均分/最高分/最低分)
7. 按平均分排名
8. 查看不及格学生名单
9. 手动保存数据到文件
0. 退出系统
#####
请输入对应功能前数字: 6
请输入制定课程名称: 高级语言程序设计
统计结果
选修该课程总人数: 2
课程最高分: 90
课程最低分: 50
课程平均分: 70

按下回车以继续.....
```

3.5 不及格学生名单

```
E:\2026春大一下\课程资 × + v
#####
请输入对应功能前数字: 6
请输入制定课程名称: 高级语言程序设计
统计结果
选修该课程总人数: 2
课程最高分: 90
课程最低分: 50
课程平均分: 70

按下回车以继续.....

***** 学生成绩管理系统 *****
请输入对应数字并回车进入对应功能.....
1. 添加学生成绩信息
2. 删除学生信息
3. 修改学生成绩
4. 查询学生成绩
5. 显示所有学生成绩
6. 成绩统计(平均分/最高分/最低分)
7. 按平均分排名
8. 查看不及格学生名单
9. 手动保存数据到文件
0. 退出系统
#####
请输入对应功能前数字: 8

===== 不及格学生名单 =====
学号: 202500130001姓名: 苏厚诚 课程: 高级语言程序设计成绩: 50

按下回车以继续.....
```

其余项目已向老师展示，概不赘述。

3.（豆包 AI 大模型生成）程序和代码说明

此行为已获得老师许可。

```
# 学生成绩管理系统课程设计报告
## 一、系统概述
本程序基于 C++面向对象思想开发学生成绩管理系统，采用`vector`存储全体学生
信息、`map`存储单名学生多门课程成绩，搭配文件持久化存储，实现学生信息增删
```

改查、成绩统计、排名、不及格筛选等完整业务功能。程序封装`Student`学生类、`Manager`成绩管理类，代码模块化清晰，具备输入容错、文件读写、数据持久化等特性，可独立编译运行，满足课程设计作业要求。

二、程序整体架构与类说明

1. Student 学生类

作用：封装单个学生所有属性与计算方法，作为数据实体。

1. 成员变量（公有）

- `string id`：学生学号，唯一标识学生；
- `string name`：学生姓名；
- `map<string, double> scores`：键为课程名、值为对应成绩，存储一名学生多门科目分数。

2. 构造函数

- 无参默认构造：适配`vector`容器初始化；
- 带参构造`Student(string sid, string sname)`：快速创建带学号姓名的学生对象。

3. 成员方法

- `double getTotal()`：遍历 map 计算该学生所有课程总分；
- `double getAvg()`：计算平均分，空课程时返回 0，避免除 0 崩溃。

2. Manager 成绩管理核心类

系统业务逻辑全部封装在此类，统一管理学生容器、文件读写、所有操作功能。

1. 成员变量

- `vector<Student> stuList`：动态数组存储全部学生，支持增删、遍历、排序；
- `string fileName`：持久化存储数据的本地文本文件名。

2. 构造函数

`Manager(string file)`：初始化文件名称，自动调用`loadData()`读取本地历史数据，程序启动自动加载上次保存信息。

3. 核心功能成员函数

函数名	功能说明
----	----
`newstu()`	添加学生/更新已有学生单科成绩，包含成绩输入容错校验 (0~100)
`deletestu()`	根据学号删除对应整条学生数据
`change()`	输入学号+课程名，修改指定科目成绩
`searchstu()`	根据学号查询单人全部成绩、总分、平均分
`showall()`	遍历 vector，打印全体学生所有信息
`countscore()`	指定课程，统计该科最高分、最低分、平均分、选课人数
`sort()`	内部冒泡排序，按平均分降序排列学生数据
`sortavg()`	调用排序函数，展示平均分排名结果
`showfail()`	遍历筛选所有 60 分以下不及格科目并输出

```
| `savedata()` | 将 vector 中全部学生数据写入本地 txt 文件持久保存 |  
| `loadData()` | 程序启动读取 txt 文件，还原所有学生数据 |
```

3. 辅助函数与主函数

1. `void menu()`：打印系统操作主菜单，展示 1~9、0 所有功能选项；
2. `main()`：程序入口，创建`Manager`对象循环展示菜单，接收用户输入并匹配对应功能，退出时自动保存数据。

三、系统完整功能清单

1. ****新增学生/更新成绩****：输入学号、姓名、课程、分数；学号重复则更新对应科目分数，自动校验成绩 0-100，非法输入循环重输；
2. ****删除学生****：输入学号，整条清除该学生所有信息，无匹配学号给出提示；
3. ****修改单科成绩****：通过学号+课程名定位，重新录入合法分数完成修改；
4. ****单人成绩查询****：输入学号，展示学号、姓名、每门课程分数、总分、平均分；
5. ****全部学生信息展示****：一次性输出所有学生学号、姓名、各科成绩、总分平均分；
6. ****单科成绩批量统计****：输入课程名，统计修读人数、最高分、最低分、平均分；无学生选课时友好提示；
7. ****平均分降序排名****：冒泡排序按平均分从高到低排序并展示排名；
8. ****不及格学生筛选****：自动检索所有低于 60 分课程，打印对应学生、科目、分数；无不及格学生给出提示；
9. ****手动保存数据****：随时将当前所有学生数据写入本地`数据库.txt`；
10. ****文件自动持久化****：程序启动自动读取历史数据，退出系统自动保存数据；
11. ****输入容错处理****：菜单数字、成绩输入错误时清空缓冲区，防止死循环。

四、编译与运行说明

环境要求

支持标准 C++11 及以上编译器：Dev-C++、VS2019/2022、MinGW g++、CodeBlocks 均可。

方法 1：MinGW g++ 命令行编译运行

1. 将完整代码复制保存为`score.cpp`；
2. 打开 cmd，切换至代码文件所在文件夹；
3. 编译指令：

```
```bash  
g++ score.cpp -o score.exe -std=c++11
```
```

4. 运行程序：

```
```bash  
score.exe
```
```

5. 运行后同目录自动生成`数据库.txt`存储学生数据，关闭程序前自动保存。

方法 2: Dev-C++ / CodeBlocks 图形化运行

1. 新建 C++ 项目，粘贴全部代码；
2. 直接点击「运行编译」按钮；
3. 控制台弹出菜单界面，根据数字输入操作。

运行注意事项

1. 程序同目录会生成`数据库.txt`，请勿手动删除、修改，否则数据丢失；
2. 成绩仅支持 0~100 数字，输入字母、负数、超 100 数值会强制重新输入；
3. 所有操作完成后按回车键返回主菜单。

五、功能说明

1. **程序启动加载界面**

描述：程序运行自动读取`数据库.txt`，控制台输出文件读取成功提示，显示当前学生总人数，随后打印主功能菜单。

2. **添加学生功能界面**

描述：选择功能 1，依次输入学号、姓名、课程、成绩；区分新增学生、已有学生更新成绩两种提示，展示非法成绩输入容错效果。

3. **单人成绩查询界面**

描述：选择功能 4，输入学生学号，打印该学生全部课程、总分、平均分。

4. **全体学生展示界面**

描述：选择功能 5，批量输出所有学生学号、姓名、各科成绩、总分平均分。

5. **课程成绩统计界面**

描述：选择功能 6，输入课程名称，输出选课人数、最高分、最低分、平均分。

6. 截图 6：平均分排名界面**

描述：选择功能 7，程序冒泡排序完成，按平均分从高到低展示排名列表。

7. **不及格学生筛选界面**

描述：选择功能 8，打印所有低于 60 分的学生及对应科目；无不及格学生则输出提示文字。

8. **删除学生、修改成绩功能界面**

描述：分别测试功能 2 删除学生、功能 3 修改单科成绩，展示操作成功/未找到学生提示。

9. **退出系统界面**

描述：输入 0 退出，程序自动保存数据至`数据库.txt`，控制台打印保存成功提示后关闭。

六、系统优缺点与总结

优点

1. 严格面向对象编程，数据与操作分离，模块化程度高，易阅读、修改；
2. 使用 vector 动态存储学生，无需固定数组长度，适配任意学生数量；map 高效管理单学生多门课程；
3. 完整实现文件持久化，重启程序不丢失数据；
4. 多层输入容错机制，避免非法输入导致程序死循环、崩溃；

5. 功能覆盖学生成绩管理全部基础需求，满足课程设计考核标准。

现存不足（可写进报告体现思考）

1. 课程名不能输入带空格文本；排序会修改原始数据；
2. 输出浮点数无小数位数限制，显示多位小数可读性较差；
3. 删除学生无二次确认，误删无法撤回；
4. 单次新增功能仅能录入一门课程，批量录入多科目操作繁琐；
5. 所有类成员变量为公有权限，未做封装保护。

总结

本次课程设计通过 C++ 类、vector、map、文件流综合实现学生成绩管理系统，掌握动态容器使用、文件读写持久化、冒泡排序、控制台交互、输入异常处理等知识点。程序功能完整稳定，可正常完成学生信息管理全部基础操作，达到本次课程设计要求。

4. 程序源码

```
#include <iostream>
#include <vector>
#include <map>
#include <fstream>
#include <string>
using namespace std;

//学生类
class Student
{
public:
    string id;//学号
    string name;//姓名
    map<string, double> scores;//map, string 是课程名, double 是分数

    //默认构造函数, 给后面 vector 用
    Student() {}
    //带参构造函数
    Student(string sid, string sname) : id(sid), name(sname) {}

    //计算单个学生总分函数
    double getTotal()
    {
        double total = 0.0;
        for (auto& p : scores)
            total += p.second;
    }
};
```

```

        return total;
    }

//计算单个学生平均分函数
double getAvg()
{
    if (scores.empty())
        return 0.0;//防止除数为0
    return getTotal() / scores.size();
}
};

//成绩管理类
class Manager
{
public:
    vector<Student>stuList; //vector 存储学生信息
    string fileName;//此变量为导出的数据文件名称

//冒泡排序
void sort()
{
    int total = stuList.size();
    for (int i = 0; i < total - 1; i++)
    {
        for (int j = 0; j < total-1- i; j++)
        {
            //以平均分作为排序依据
            if (stuList[j].getAvg() < stuList[j + 1].getAvg())
            {
                Student t = stuList[j];
                stuList[j] = stuList[j + 1];
                stuList[j + 1] = t;
            }
        }
    }
}

//构造函数：初始化文件名，加载文件数据
Manager(string file) : fileName(file)
{

```



```

        loadData();
    }

//功能：添加学生及成绩信息
void newstu()
{
    string id, name, course;
    double score;
    cout << "==== 添加学生及成绩信息 =====>> endl;
    cout << "请输入学号：";
    cin >> id;
    cout << "请输入姓名：";
    cin >> name;
    cout << "请输入课程名称：";
    cin >> course;
    cout << "请输入成绩(成绩范围：0-100)：";

    //输入容错机制
    while (!(cin >> score) || score < 0 || score > 100)
    {
        cin.clear();//清理错误标记
        cin.ignore(1024, '\n');//忽略缓冲区最多 1024 字符，直到碰见回
车键，防止卡住
        cout << "输入有误，成绩范围应是 0-150。请重新输入：";
    }

    bool a = 0;
    for (auto& stu : stuList)//auto 自动匹配 Student, stu, 其在遍历容
器的过程中每次取容器中其中一个学生的信息
    {
        if (stu.id == id)
        {
            a = 1;
            stu.scores[course] = score;
            cout << "该学生存在，已对现有成绩更新完毕">> endl;//自动
更新成绩
            break;
        }
    }
    if (a==0)//不存在的学生，那就是新增喽
    {
        Student newStu(id, name);
    }
}

```

```

        newStu.scores[course] = score;
        stuList.push_back(newStu);
        cout << "新学生学生信息添加完毕" << endl;
    }
}

```

//功能：删除指定学生信息

```
void deletestu()
```

```

{
    string id;
    cout << "==== 删除学生信息 =====" << endl;
    cout << "请输入该生学号：";
    cin >> id;

    for (auto it = stuList.begin(); it != stuList.end(); it++)//根据
学号检索
    {
        if (it->id == id)
        {
            stuList.erase(it);
            cout << "该生已删除" << endl;
            return;
        }
    }
    cout << "未查询到学生" << endl;
}

```

//功能：修改学生课程成绩

```
void change()
```

```

{
    string id, course;
    double newScore;
    cout << "==== 修改学生成绩 =====" << endl;
    cout << "请输入该生学号：";
    cin >> id;
    cout << "请输入要修改成绩的课程名：";
    cin >> course;

    for (auto& stu : stuList)
    {
        if (stu.id == id)
        {

```

```

        if (stu.scores.find(course) == stu.scores.end())//前面
map 容器形式的 scores 的自带函数 find
    {
        cout << "该学生无此课程成绩！" << endl;
        return;
    }
    cout << "请输入新成绩(成绩范围：0-100)：";
    while (!(cin >> newScore) || newScore < 0 || newScore >
100)
    {
        cin.clear();
        cin.ignore(1024, '\n');
        cout << "输入无效！请重新输入：";//依旧防止意外发生，
同上
    }
    stu.scores[course] = newScore;
    cout << "成绩修改成功！" << endl;
    return;
}
}
cout << "未查询到该学号学生！" << endl;
}

//功能：查询单个学生成绩
void searchstu()
{
    string id;
    cout << "==== 查询学生成绩 =====" << endl;
    cout << "请输入学号：";
    cin >> id;

    for (auto& stu : stuList)
    {
        if (stu.id == id)
        {
            cout << "---查询成功---" << endl;
            cout<< "学号：" << stu.id << "姓名：" << stu.name <<
endl;

            cout<< "课程成绩：" << endl;
            for (auto& p : stu.scores)
            {
                cout << p.first << "： " << p.second << "； " <<

```

```

endl;//first 对应的是科目，second 对应的是成绩
        }
        cout << "总分：" << stu.getTotal() << "平均分：" <<
stu.getAvg() << endl;
        return;
    }
}
cout << "未查询到该学号学生！" << endl;
}

//功能：展示全部学生成绩
void showall()
{
    if (stuList.empty())
    {
        cout << "暂无学生数据，请检查是否已录入学生" << endl;
        return;//异常处理，防止空集
    }
    cout << "\n===== 全体学生成绩
===== " << endl;
    for (auto& stu : stuList)
    {
        cout << "学号：" << stu.id << " 姓名：" << stu.name;
        cout << "总分：" << stu.getTotal() << " 平均分：" <<
stu.getAvg() << endl;
        cout << " 各课程成绩：" ;
        for (const auto& p : stu.scores)
            cout << p.first << "：" << p.second << "； ";
        cout << endl;
    }
}

//功能：统计制定课程所有学生的最高分、最低分、平均分
void countscore()
{
    if (stuList.empty())
    {
        cout << "暂无学生数据，请检查是否已录入学生" << endl;
        return;
    }

    string classes;

```

```
cout << "请输入制定课程名称：";
cin >> classes;

double numscores = 0.0;
double max = 0.0, min = 100.0;
int studentnum = 0;

// 遍历所有学生
for (auto& stu : stuList)
{
    // 查找该学生是否具有该课程成绩
    auto p = stu.scores.find(classes);
    if (p == stu.scores.end())
        continue; // 无该课程则跳过

    double tempscore = p->second;
    numscores = numscores + tempscore;
    studentnum++;

    if (tempscore > max)
    {
        max = tempscore;
    }

    if (tempscore < min)
    {
        min = tempscore;
    }
}

// 判断是否存在该课程成绩
if (studentnum == 0)
{
    cout << "暂无学生修读此课程" << endl;
    return;
}

double avg = numscores / studentnum;
cout << "统计结果" << endl;
cout << "选修该课程总人数：" << studentnum << endl;
cout << "课程最高分：" << max << endl;
cout << "课程最低分：" << min << endl;
```

```

        cout << "课程平均分：" << avg << endl;
    }

//功能：平均分降序排名
void sortavg()
{
    if (stuList.empty())
    {
        cout << "暂无数据，请检查是否录入数据" << endl;
        return;
    }
    sort();
    cout << "已按平均分从高到低完成排名！" << endl;
    showall();
}

//功能：挂科学生鞭尸环节
void showfail()
{
    bool a = 0;
    cout << "\n==== 不及格学生名单 =====" << endl;
    for (auto& stu :stuList)
    {
        for (auto& p :stu.scores)
        {
            if (p.second < 60)
            {
                a = 1;
                cout << "学号：" << stu.id << "姓名：" << stu.name;
                cout << " 课程：" << p.first << "成绩：" << p.second
<< endl;

                break;
            }
        }
    }
    if (a==0)
        cout << "暂无不及格学生！" << endl;
}

//功能：保存数据到文件（纯vibe coding）
void savedata()
{

```

```

        ofstream out(fileName, ios::out);
        if (!out)
        {
            cout << "文件打开异常" << endl;
            return;
        }
        for (auto& stu : stuList)
        {
            out << stu.id << " " << stu.name << " " << stu.scores.size()
<< " ";
            for (auto& p : stu.scores)
                out << p.first << " " << p.second << " ";
            out << endl;
        }
        out.close();
        cout << "已保存数据到文件" << endl;
    }

//功能：从文件加载数据（纯 vibe coding）
void loadData()
{
    ifstream in(fileName, ios::in);
    if (!in)
    {
        cout << "数据文件不存在，系统将新建文件" << endl;
        return;
    }
    stuList.clear();
    string id, name, course;
    int courseNum;
    double score;
    while (in >> id >> name >> courseNum) //从文件中连续读取学号、姓名、课程门数，读取失败或没有时自动终止
    {
        Student stu(id, name);
        for (int i = 0; i < courseNum; i++)
        {
            in >> course >> score;
            stu.scores[course] = score;
        }
        stuList.push_back(stu); //把此时的临时 stu 对象（刚读取的）添加到 vector 末尾，即新增成员
    }
}

```

```

    }
    in.close();
    cout << "文件数据读取完成, 当前共" << stuList.size() << "名学生"
<< endl;
    }
};

//主菜单函数
void menu()
{
    cout << "\n***** 学生成绩管理系统 *****" << endl;
    cout << "请输入对应数字并回车进入对应功能....." << endl;
    cout << "1. 添加学生成绩信息" << endl;
    cout << "2. 删除学生信息" << endl;
    cout << "3. 修改学生成绩" << endl;
    cout << "4. 查询学生成绩" << endl;
    cout << "5. 显示所有学生成绩" << endl;
    cout << "6. 成绩统计(平均分/最高分/最低分)" << endl;
    cout << "7. 按平均分排名" << endl;
    cout << "8. 查看不及格学生名单" << endl;
    cout << "9. 手动保存数据到文件" << endl;
    cout << "0. 退出系统" << endl;
    cout << "#####" << endl;
    cout << "请输入对应功能前数字: ";
}

int main()
{
    Manager manager("数据库.txt");
    int choice;

    while (1)
    {
        menu();
        while (!(cin >> choice))
        {
            cin.clear();//还原 cin
            cin.ignore(1024, '\n');//清空缓冲区所有残留字符直到换行, 避免
死循环
            cout << "输入无效, 请重新输入 0-9 间的阿拉伯数字编号: ";//输入
异常处理, 同上
        }
    }
}

```



```

switch (choice)
{
case 1: manager.newstu(); break;
case 2: manager.deletestu(); break;
case 3: manager.change(); break;
case 4: manager.searchstu(); break;
case 5: manager.showall(); break;
case 6: manager.countscore(); break;
case 7: manager.sortavg(); break;
case 8: manager.showfail(); break;
case 9: manager.savedata(); break;
case 0:
    cout << "退出系统，数据已自动保存！" << endl;
    manager.savedata();
    return 0;
default:
    cout << "编号错误，请重新选择" << endl;
}
cout << "\n 按下回车以继续.....";
cin.ignore(1024, '\n');//清空防止之前输入的回车被读取导致用户一脸
懵

    cin.get();
}
return 0;
}

```

5. AI 使用情况说明

本项目框架和部分函数为本人原创；部分功能函数依赖 AI 编写（提示词为本人原创）；代码注释全部为本人原创；本报告由人工和 AI 总结而成。